Nam Tu - 153076622 - Frequent Itemsets

Task 3.1

The first 2 subtasks were done with a very simple implementation of an algorithm to find frequent items according to the instructions. The final subtask was done with the condition in subtask a. The Apriori was implemented to generate n-itemsets, generating their supports and then the candidate itemsets for (n+1)-itemsets until there are no more candidate itemsets.

Task 3.1a

In [2]:
```
num_items = 100
num_baskets = 100
support_threshold = 5

#computing support for items
support = {}
for i in range(1,num_items+1):
    count = 0
    for b in range(1,num_baskets+1):
        if b%i == 0:
            count += 1
    support[i] = count


frequent_items = [i for i in range(1, num_items + 1) if support[i] >= support_th


print("\nFrequent items (support >= 5):")
print(frequent_items)
```

```
Frequent items (support >= 5):
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
```

In [3]:
```
#computing support for items
support = {}
for i in range(1,num_items+1):
    count = 0
    for b in range(1,num_baskets+1):
        if i%b == 0:
            count += 1
    support[i] = count


frequent_items = [i for i in range(1, num_items + 1) if support[i] >= support_th
```

```
print("\nFrequent items (support >= 5):")
print(frequent_items)
```

```
Frequent items (support >= 5):
[12, 16, 18, 20, 24, 28, 30, 32, 36, 40, 42, 44, 45, 48, 50, 52, 54, 56, 60, 63,
64, 66, 68, 70, 72, 75, 76, 78, 80, 81, 84, 88, 90, 92, 96, 98, 99, 100]
```

Task 3.1b

Algorithm constructed with support from Copilot

1. Generate baskets and get their supports

2. Generate L1 (frequent 1-itemsets)

3. Generate C2 (candidate doubles)

4. Compute supports for C2

5. Generate L2

6. Generate C3

7. Compute supports for C3

8. Generate C4

9. Code stops because no new frequent itemsets are found

In [ ]:

```
from itertools import combinations

#Generate baskets
def generate_baskets(n=100):
    baskets = []
    for b in range(1, n + 1):
        basket = set()
        for i in range(1, n + 1):
            if b % i == 0:
                basket.add(i)
        baskets.append(basket)
    return baskets

#Count support
def get_support(itemset, baskets):
    count = 0
    for basket in baskets:
        if itemset.issubset(basket):
            count += 1
    return count

#Apriori algorithm
def apriori(baskets, min_support):
    # Generate 1-itemsets
```

```python
    items = set()
    for basket in baskets:
        items |= basket

    freq_itemsets = []
    L1 = []

    for item in items:
        support = get_support({item}, baskets)
        if support >= min_support:
            L1.append(frozenset([item]))

    current_L = L1
    k = 2


    #Generate C2 then L2 then C3 then L3 so on and son
    while current_L:
        freq_itemsets.extend(current_L)

        # Generate candidates
        candidates = set()
        for a, b in combinations(current_L, 2):
            union = a | b
            if len(union) == k:
                candidates.add(union)

        # Prune step
        pruned_candidates = set()
        for candidate in candidates:
            all_subsets_frequent = True
            for subset in combinations(candidate, k - 1):
                if frozenset(subset) not in current_L:
                    all_subsets_frequent = False
                    break
            if all_subsets_frequent:
                pruned_candidates.add(candidate)

        # Count support
        next_L = []
        for candidate in pruned_candidates:
            support = get_support(candidate, baskets)
            if support >= min_support:
                next_L.append(candidate)

        current_L = next_L
        k += 1

    return freq_itemsets

# Run everything
baskets = generate_baskets(100)
min_support = 5

frequent_itemsets = apriori(baskets, min_support)

# Print results
print(", ".join("{" + ", ".join(map(str, sorted(itemset))) + "}" for itemset in
```

```
{1}, {2}, {3}, {4}, {5}, {6}, {7}, {8}, {9}, {10}, {11}, {12}, {13}, {14}, {15},
{16}, {17}, {18}, {19}, {20}, {2, 8}, {2, 3}, {7, 14}, {1, 10}, {2, 9}, {3, 4},
{1, 14}, {2, 4}, {1, 19}, {10, 20}, {5, 15}, {5, 10}, {1, 16}, {2, 7}, {1, 9},
{2, 10}, {2, 18}, {1, 11}, {2, 5}, {2, 12}, {4, 8}, {4, 10}, {1, 15}, {4, 5}, {1,
7}, {2, 16}, {3, 12}, {3, 5}, {1, 6}, {3, 18}, {2, 14}, {4, 20}, {4, 12}, {3, 9},
{2, 6}, {1, 20}, {3, 6}, {1, 18}, {1, 17}, {1, 12}, {1, 5}, {8, 16}, {9, 18}, {1,
2}, {5, 20}, {1, 4}, {4, 6}, {4, 16}, {6, 9}, {1, 8}, {6, 12}, {6, 18}, {1, 3},
{2, 20}, {1, 13}, {3, 15}, {2, 9, 18}, {1, 2, 9}, {1, 6, 12}, {2, 3, 4}, {1, 6, 1
8}, {2, 6, 9}, {1, 3, 5}, {1, 3, 18}, {2, 4, 12}, {1, 10, 20}, {2, 4, 16}, {2, 7,
14}, {5, 10, 20}, {1, 5, 20}, {1, 5, 15}, {1, 2, 10}, {3, 4, 12}, {2, 4, 8}, {1,
4, 8}, {3, 6, 9}, {1, 4, 10}, {1, 2, 14}, {1, 4, 6}, {2, 4, 6}, {1, 5, 10}, {2,
3, 9}, {1, 2, 20}, {2, 10, 20}, {1, 2, 7}, {1, 4, 20}, {4, 10, 20}, {1, 7, 14},
{3, 4, 6}, {2, 6, 12}, {1, 2, 8}, {4, 6, 12}, {1, 4, 12}, {1, 2, 3}, {3, 5, 15},
{1, 2, 6}, {1, 3, 4}, {2, 6, 18}, {2, 3, 12}, {2, 8, 16}, {1, 8, 16}, {2, 3, 18},
{1, 2, 18}, {2, 3, 6}, {1, 2, 5}, {1, 3, 9}, {2, 5, 10}, {1, 3, 15}, {1, 4, 5},
{1, 2, 12}, {4, 5, 10}, {3, 9, 18}, {2, 4, 20}, {4, 8, 16}, {2, 5, 20}, {4, 5, 2
0}, {2, 4, 10}, {1, 2, 4}, {1, 3, 6}, {3, 6, 18}, {2, 4, 5}, {1, 6, 9}, {6, 9, 1
8}, {3, 6, 12}, {1, 4, 16}, {1, 2, 16}, {1, 3, 12}, {1, 9, 18}, {1, 3, 4, 12},
{3, 4, 6, 12}, {1, 4, 5, 10}, {2, 3, 6, 12}, {1, 2, 5, 10}, {1, 4, 8, 16}, {1, 2,
4, 20}, {1, 2, 9, 18}, {2, 3, 4, 6}, {1, 3, 4, 6}, {1, 6, 9, 18}, {1, 2, 4, 12},
{1, 3, 5, 15}, {1, 2, 8, 16}, {1, 4, 5, 20}, {1, 2, 5, 20}, {1, 2, 4, 8}, {2, 3,
4, 12}, {1, 2, 4, 10}, {2, 3, 6, 9}, {1, 3, 6, 18}, {1, 2, 4, 6}, {1, 3, 6, 12},
{2, 3, 9, 18}, {1, 3, 9, 18}, {1, 2, 3, 9}, {1, 2, 4, 5}, {2, 4, 10, 20}, {1, 5,
10, 20}, {1, 2, 7, 14}, {2, 4, 5, 20}, {1, 3, 6, 9}, {3, 6, 9, 18}, {2, 4, 6, 1
2}, {1, 2, 3, 4}, {1, 2, 6, 9}, {2, 6, 9, 18}, {1, 2, 4, 16}, {2, 3, 6, 18}, {1,
4, 6, 12}, {1, 2, 3, 12}, {1, 2, 6, 12}, {1, 2, 6, 18}, {2, 4, 5, 10}, {1, 2, 3,
18}, {1, 4, 10, 20}, {2, 4, 8, 16}, {1, 2, 10, 20}, {4, 5, 10, 20}, {2, 5, 10, 2
0}, {1, 2, 3, 6}, {1, 2, 3, 6, 18}, {1, 2, 5, 10, 20}, {1, 4, 5, 10, 20}, {1, 2,
3, 4, 6}, {1, 2, 3, 6, 12}, {1, 3, 4, 6, 12}, {1, 2, 4, 10, 20}, {1, 2, 4, 5, 2
0}, {1, 2, 4, 6, 12}, {1, 2, 3, 4, 12}, {1, 2, 6, 9, 18}, {1, 2, 3, 9, 18}, {2,
3, 6, 9, 18}, {1, 2, 3, 6, 9}, {1, 2, 4, 5, 10}, {2, 3, 4, 6, 12}, {2, 4, 5, 10,
20}, {1, 3, 6, 9, 18}, {1, 2, 4, 8, 16}, {1, 2, 3, 6, 9, 18}, {1, 2, 3, 4, 6, 1
2}, {1, 2, 4, 5, 10, 20}
```

Task 3.2

Task 3.2.1

FP-Growth (Frequent-Pattern-Growth) Algorithm also mines for frequent itemsets but unlike Aprior, it is not as computationally costly by compressing data in to a FP-tree and extracting patterns directly from it only with 2 database scans

1. Data compression: Compresses data into a FP-tree that stores information about itemsets and their frequencies without needing to generate candidate sets like Apriori

2. Tree Mining: Examines tree to find patterns that appear based on minimum threshold, the algorithm does this via breaking it down into smaller "conditional" trees for each item

3. Generate patterns: Generate frequent itemsets and rules describing relationships

Task 3.2.2

These files contain 1,000,209 anonymous ratings of approximately 3,900 movies made by 6,040 MovieLens users who joined MovieLens in 2000.

At only(?) 35% threshold for minimum support, the list of frequent itemsets only has 2 members left. It can be seen from the few test thresholds that for a large dataset, a few percentage points can significantly alter the amount of frequent itemsets. Additionally, computing these sets with FPGrowth took a significantly shorter amount of time than compared to Aprior in the last task.

In [1]:
```
import os
import pyspark
from pyspark.sql import *
sc = pyspark.SparkContext("local", "FrequentItemsets")
os.system('wget http://files.grouplens.org/datasets/movielens/ml-1m.zip; unzip m
```

Out[1]: 1

In [ ]:
```
from pyspark.mllib.fpm import FPGrowth
lines = sc.textFile("ml-1m/movies.dat")
movienames = lines.map(lambda l: l.split("::")). \
    map(lambda p: ( int(p[0]), p[1] ) ).collect()
#convert to dictionary for printing lookup
movienames = dict( movienames )
#read ratings file as lines of text, assuming no errors on data
lines = sc.textFile("ml-1m/ratings.dat")
#group user ratings into (user, list of rated movie ids)
usermovies = lines.map(lambda l: l.split("::")). \
    map(lambda p: ( int(p[0]), int(p[1]) ) ).groupByKey()
#take only list part
usermovies = usermovies.map(lambda p: list(p[1]))
usercount = usermovies.count()
model = FPGrowth.train(usermovies, minSupport=0.2, numPartitions=10)
result = model.freqItemsets()
```

In [7]:
```
for x in result.collect():
    keys = x[0]
    if len(keys)>4:
        fpairs = []
        for y in keys:
            fpairs.append(movienames[y])
        print(fpairs)
        print("Found in " + str(x[1]) + " user of " + str(usercount) + "\n")
```

```
['Men in Black (1997)', 'Jurassic Park (1993)', 'Star Wars: Episode VI - Return o
f the Jedi (1983)', 'Star Wars: Episode V - The Empire Strikes Back (1980)', 'Sta
r Wars: Episode IV - A New Hope (1977)']
Found in 1217 user of 6040

['Men in Black (1997)', 'Matrix, The (1999)', 'Star Wars: Episode VI - Return of
the Jedi (1983)', 'Star Wars: Episode V - The Empire Strikes Back (1980)', 'Star
Wars: Episode IV - A New Hope (1977)']
Found in 1219 user of 6040

['Matrix, The (1999)', 'Jurassic Park (1993)', 'Star Wars: Episode VI - Return of
the Jedi (1983)', 'Star Wars: Episode V - The Empire Strikes Back (1980)', 'Star
Wars: Episode IV - A New Hope (1977)']
Found in 1222 user of 6040

['Matrix, The (1999)', 'Terminator 2: Judgment Day (1991)', 'Star Wars: Episode V
I - Return of the Jedi (1983)', 'Star Wars: Episode V - The Empire Strikes Back
(1980)', 'Star Wars: Episode IV - A New Hope (1977)']
Found in 1282 user of 6040

['Terminator, The (1984)', 'Terminator 2: Judgment Day (1991)', 'Star Wars: Episo
de VI - Return of the Jedi (1983)', 'Star Wars: Episode V - The Empire Strikes Ba
ck (1980)', 'Star Wars: Episode IV - A New Hope (1977)']
Found in 1252 user of 6040

['Terminator, The (1984)', 'Matrix, The (1999)', 'Terminator 2: Judgment Day (199
1)', 'Star Wars: Episode V - The Empire Strikes Back (1980)', 'Star Wars: Episode
IV - A New Hope (1977)']
Found in 1220 user of 6040
```

The code looks at reviews by users and generates frequent itemsets of movies that have been reviewed.

40% support threshold - No results

In [9]:

```
lines = sc.textFile("ml-1m/movies.dat")
moviesnames = lines.map(lambda l: l.split("::")). \
    map(lambda p: ( int(p[0]), p[1] ) ).collect()
#convert to dictionary for printing lookup
moviesnames = dict( moviesnames )
#read ratings file as lines of text, assuming no errors on data
lines = sc.textFile("ml-1m/ratings.dat")
#group user ratings into (user, list of rated movie ids)
usermovies = lines.map(lambda l: l.split("::")). \
    map(lambda p: ( int(p[0]), int(p[1]) ) ).groupByKey()
#take only list part
usermovies = usermovies.map(lambda p: list(p[1]))
usercount = usermovies.count()
model = FPGrowth.train(usermovies, minSupport=0.4, numPartitions=10)
result = model.freqItemsets()

for x in result.collect():
    keys = x[0]
    if len(keys)>1:
        fpairs = []
```

```
        for y in keys:
            fpairs.append(movienames[y])
        print(fpairs)
        print("Found in " + str(x[1]) + " user of " + str(usercount) + "\n")
```

30% support threshold - List significantly reduced

In [11]:
```
lines = sc.textFile("ml-1m/movies.dat")
movienames = lines.map(lambda l: l.split("::")). \
    map(lambda p: ( int(p[0]), p[1] ) ).collect()
#convert to dictionary for printing lookup
movienames = dict( movienames )
#read ratings file as lines of text, assuming no errors on data
lines = sc.textFile("ml-1m/ratings.dat")
#group user ratings into (user, list of rated movie ids)
usermovies = lines.map(lambda l: l.split("::")). \
    map(lambda p: ( int(p[0]), int(p[1]) ) ).groupByKey()
#take only list part
usermovies = usermovies.map(lambda p: list(p[1]))
usercount = usermovies.count()
model = FPGrowth.train(usermovies, minSupport=0.3, numPartitions=10)
result = model.freqItemsets()

for x in result.collect():
    keys = x[0]
    if len(keys)>1:
        fpairs = []
        for y in keys:
            fpairs.append(movienames[y])
        print(fpairs)
        print("Found in " + str(x[1]) + " user of " + str(usercount) + "\n")
```

```
['Men in Black (1997)', 'Jurassic Park (1993)']
Found in 1939 user of 6040

['Men in Black (1997)', 'Terminator 2: Judgment Day (1991)']
Found in 1855 user of 6040

['Raiders of the Lost Ark (1981)', 'Star Wars: Episode IV - A New Hope (1977)']
Found in 1972 user of 6040

['Raiders of the Lost Ark (1981)', 'Star Wars: Episode V - The Empire Strikes Bac
k (1980)']
Found in 1999 user of 6040

['Star Wars: Episode V - The Empire Strikes Back (1980)', 'American Beauty (199
9)']
Found in 1829 user of 6040

['Star Wars: Episode V - The Empire Strikes Back (1980)', 'Star Wars: Episode IV
- A New Hope (1977)']
Found in 2355 user of 6040

['Fargo (1996)', 'American Beauty (1999)']
Found in 1856 user of 6040

['Being John Malkovich (1999)', 'American Beauty (1999)']
Found in 1884 user of 6040

['Star Wars: Episode VI - Return of the Jedi (1983)', 'Star Wars: Episode IV - A
New Hope (1977)']
Found in 2113 user of 6040

['Star Wars: Episode VI - Return of the Jedi (1983)', 'Star Wars: Episode V - The
Empire Strikes Back (1980)']
Found in 2228 user of 6040

['Star Wars: Episode VI - Return of the Jedi (1983)', 'Star Wars: Episode V - The
Empire Strikes Back (1980)', 'Star Wars: Episode IV - A New Hope (1977)']
Found in 1926 user of 6040

['Jurassic Park (1993)', 'Star Wars: Episode IV - A New Hope (1977)']
Found in 1824 user of 6040

['Jurassic Park (1993)', 'Star Wars: Episode V - The Empire Strikes Back (1980)']
Found in 1814 user of 6040

['Terminator 2: Judgment Day (1991)', 'Star Wars: Episode IV - A New Hope (197
7)']
Found in 1889 user of 6040

['Terminator 2: Judgment Day (1991)', 'Star Wars: Episode V - The Empire Strikes
Back (1980)']
Found in 1911 user of 6040

['Terminator 2: Judgment Day (1991)', 'Jurassic Park (1993)']
Found in 1909 user of 6040

['Matrix, The (1999)', 'Star Wars: Episode IV - A New Hope (1977)']
Found in 1908 user of 6040

['Matrix, The (1999)', 'Star Wars: Episode V - The Empire Strikes Back (1980)']
```

```
Found in 1920 user of 6040

['Matrix, The (1999)', 'Jurassic Park (1993)']
Found in 1834 user of 6040

['Matrix, The (1999)', 'Terminator 2: Judgment Day (1991)']
Found in 1978 user of 6040

['Back to the Future (1985)', 'Star Wars: Episode IV - A New Hope (1977)']
Found in 1845 user of 6040

['Back to the Future (1985)', 'Star Wars: Episode V - The Empire Strikes Back (19
80)']
Found in 1905 user of 6040

['Silence of the Lambs, The (1991)', 'American Beauty (1999)']
Found in 1847 user of 6040
```

35% support threshold - List has 2 itemsets left

In [12]:

```
lines = sc.textFile("ml-1m/movies.dat")
moviesnames = lines.map(lambda l: l.split("::")). \
    map(lambda p: ( int(p[0]), p[1] ) ).collect()
#convert to dictionary for printing lookup
moviesnames = dict( moviesnames )
#read ratings file as lines of text, assuming no errors on data
lines = sc.textFile("ml-1m/ratings.dat")
#group user ratings into (user, list of rated movie ids)
usermovies = lines.map(lambda l: l.split("::")). \
    map(lambda p: ( int(p[0]), int(p[1]) ) ).groupByKey()
#take only list part
usermovies = usermovies.map(lambda p: list(p[1]))
usercount = usermovies.count()
model = FPGrowth.train(usermovies, minSupport=0.35, numPartitions=10)
result = model.freqItemsets()

for x in result.collect():
    keys = x[0]
    if len(keys)>1:
        fpairs = []
        for y in keys:
            fpairs.append(moviesnames[y])
        print(fpairs)
        print("Found in " + str(x[1]) + " user of " + str(usercount) + "\n")
```

```
['Star Wars: Episode V - The Empire Strikes Back (1980)', 'Star Wars: Episode IV
- A New Hope (1977)']
Found in 2355 user of 6040

['Star Wars: Episode VI - Return of the Jedi (1983)', 'Star Wars: Episode V - The
Empire Strikes Back (1980)']
Found in 2228 user of 6040
```